

PotensicProxy_TAF - Projektdokumentation

Kamera-Log-0x39-Dekodierung - v1.5

Projekt	PotensicProxy_TAF
Dokumentenart	Projektdokumentation / Implementierungsnachweis
Erstellungsdatum	2026-10-02
Bearbeiter	OpenAI / ChatGPT
Version	v1.5
Status	Arbeitsstand - umgesetzt; Geräte-Regressionsprüfung offen
Änderungsdatum	2026-10-02
Ablageort	Projektordner DOCUMENTATION/
Referenzen	PotensicPro.zip; 2026-10-02_23-58-07_log.xml; Protokollreferenz v0.6; Vorgängerversion v1.4

1. Anlass und Ziel

Die v1.4-Implementierung klassifizierte Kamera-Command 0x39 bereits korrekt als asynchronen Kamera-Log-Datensatz, interpretierte den Inhalt jedoch noch pauschal als UTF-8-Text. Reale Live-Logs zeigten dadurch unlesbare Zeichenfolgen bei CAMERA/LINUX und Ersatzzeichen bei CAMERA/GIMBAL. Ziel von v1.5 ist die bytgenaue Auswertung entsprechend der Original-App und dem beigefügten USB-Paketmitschnitt, ohne unbestätigte Semantik zu erfinden.

2. Quellbefund aus der Original-App

Die Original-App PotensicPro enthält die Klassen CameraLogData, CameraLogRecorder und UsbCameraHandler. UsbCameraHandler behandelt Command 57 (0x39) vor der normalen Statuscode-Auswertung und erzeugt unmittelbar CameraLogData. Damit ist bestätigt, dass 0x39 kein gewöhnlicher Command/Result-Response ist.

2.1 CameraLogData-Struktur

Offset relativ zu 0x39	Länge	Feld	Beleg / Bedeutung
+0	1	command	0x39, bereits durch UsbCameraHandler dispatcht
+1	1	source	0=Linux, 1=LiteOS, 2=Gimbal
+2	2	payloadLength	uint16 little endian
+4	payloadLength	payload	exakt diese Anzahl Rohbytes

CameraLogRecorder schreibt die Payload der drei Quellen bytgenau in getrennte Dateien: CAMLinux, CAMLiteos und gimbal. Die Original-App wendet an dieser Stelle keinen generischen Textdecoder auf die Rohdaten an. Dieser Befund erklärt die fehlerhafte v1.4-Ausgabe: Source-/Längenfelder und binäre Gimbal-Daten dürfen nicht als Text interpretiert werden.

3. Abgleich mit dem USB-Paketlog

Der beigefügte Mitschnitt 2026-10-02_23-58-07_log.xml enthält 1000 gepufferte Pakete. Eine strukturelle Auswertung aller darin enthaltenen Kamera-Log-Frames ergab 37 Pakete mit FE 0x05, innerer Function 0x0020 und Command 0x39. Davon sind 32 Source 0 (Linux) und 5 Source 2 (Gimbal). Source 1 (LiteOS) kommt in diesem Mitschnitt nicht vor. Bei allen 37 Paketen stimmt die deklarierte 16-Bit-Länge mit der tatsächlich vorhandenen Source-Payload überein.

3.1 Linux: XOR 0x55 bestätigt

Für alle 32 Source-0-Payloads ergibt die byteweise Operation `decodedByte = rawByte XOR 0x55` lesbaren ASCII-Diagnosetext. Damit ist die Ursache des bisherigen Zeichensalats konkret bestimmt. Beispiele aus dem Mitschnitt:

- [12:19:15:827]:DS_GetEVString[30]:Work Mode State is Progress
- [12:19:15:827]:camera_soc_temperature_monitor(3694):dsl[soc_temperature 78.0]
- [12:19:15:684]:observer_list_traverse(235):dsl[cpu_SyncThread 0.0]
- [12:19:15:684]:monitor_cpu_proc(252):dsl[cpu1_sum 23.3, cpu1_usr 17.6, cpu1_sys 5.6, cpu1_io 0.0]

3.2 Gimbal: Binärdaten

Die Source-2-Pakete besitzen binäre Nutzdaten. Ein Beispiel deklariert `0x0100 = 256 Payloadbytes`. Eine Textdekodierung erzeugt deshalb zwangsläufig Ersatzzeichen und Mehrzeilenmüll. v1.5 behandelt Gimbal-Logs bewusst als Binärstrom und zeigt im Live-Log nur Länge und eine begrenzte Hex-Vorschau.

4. Umsetzung v1.5

- 0x39 bleibt vor der generischen Kamera-Statusauswertung abgefangen.
- Der Post-Command-Header wird als `source + uint16LE payloadLength` ausgewertet.
- Nur exakt `payloadLength Bytes` werden als Source-Payload verwendet; eine zu große deklarierte Länge wird als truncated verworfen.
- Linux (Source 0) wird byteweise mit XOR 0x55 dekodiert.
- CR/LF/TAB und sonstige Steuerbytes werden escaped, damit ein Kamera-Datensatz genau einen Live-System-Log-Eintrag belegt.
- Gimbal (Source 2) wird nicht textdekodiert; angezeigt werden `declared length` und maximal 48 Rohbytes als Hex-Vorschau.
- LiteOS (Source 1) bleibt zunächst Raw-Hex, weil der vorliegende Mitschnitt keinen Source-1-Datensatz enthält und daher keine Kodierung bestätigt.
- Unerwartete Bytes hinter der deklarierten Payload werden als Warnung protokolliert.

5. Geänderte Dateien

- `webui/src/services/CameraMediaService.ts`
- `PROTOCOL.md`
- `README.md`
- `DOCUMENTATION/2026-10-02 - PotensicProxy_TAF - Projektdokumentation - Kamera-Log-0x39-Dekodierung - v1.5.docx`
- `DOCUMENTATION/2026-10-02 - PotensicProxy_TAF - Projektdokumentation - Kamera-Log-0x39-Dekodierung - v1.5.pdf`
- `PATCH_CAMERA_LOG_0X39_DECODE_2026-10-02_v1.5.patch`
- `NUR_AENDERUNGEN_IN_ORIGINALSTRUKTUR_2026-10-02_CAMERA_LOG_0X39_DECODE_v1.5.md`
- `COMMIT_MESSAGE_CAMERA_LOG_0X39_DECODE_2026-10-02_v1.5.txt`

6. Validierung

Prüfung	Status	Ergebnis
TypeScript/Vue Typprüfung	PASS	<code>webui/node_modules/.bin/vue-tsc --noEmit</code> ohne Fehler.
0x39-Strukturprüfung im USB-Log	PASS	37/37 Frames mit gültiger deklarerter Payload-Länge; 32 Linux, 5 Gimbal.
Linux XOR 0x55	PASS	Source-0-Payloads werden zu lesbaren Kamera-Diagnosetexten.

Prüfung	Status	Ergebnis
Vite Produktionsbuild	BLOCKIERT	TypeScript läuft durch; Vite kann in der gelieferten node_modules-Struktur das optionale Linux-Rollup-Binary @rollup/rollup-linux-x64-gnu nicht laden.
Reale Geräte-Regressionsprüfung	OFFEN	Nach Installation auf Android prüfen, dass CAMERA/LINUX Klartext und CAMERA/GIMBAL eine kompakte Binärvorschau zeigt.

7. Erwartetes Laufzeitverhalten

Nach der Aktualisierung sollen Linux-Kamera-Logs nicht mehr als Zeichenfolgen wie "dgoelod..." erscheinen, sondern als lesbare CAMERA/LINUX-Diagnosezeilen. Gimbal-Pakete sollen keine Folge von Unicode-Ersatzzeichen mehr erzeugen, sondern z. B. als "[CAMERA/GIMBAL] len=256 raw=CD 00 ..." erscheinen. Dadurch bleibt der 1000-Einträge-Rolling-Buffer wesentlich aussagekräftiger und ein binärer Kamera-Datensatz kann nicht mehr mehrere sichtbare Logzeilen verbrauchen.

8. Grenzen

Die XOR-0x55-Regel ist durch die 32 Linux-Payloads im beigefügten Mitschnitt validiert. Für LiteOS Source 1 liegt in diesem Mitschnitt kein Beispiel vor; v1.5 nimmt deshalb bewusst keine unbestätigte XOR- oder Textkodierung an. Die Original-App speichert alle drei Source-Payloads roh. Eine zusätzliche separate Raw-Camera-Log-Datei in TAF ist in v1.5 nicht eingeführt; die vollständigen Rohbytes bleiben weiterhin im USB-Paketlog erhalten.

9. Empfohlene Geräteabnahme

- App starten und prüfen, dass eingehende 0x39-Linux-Datensätze als lesbare [CAMERA/LINUX]-Zeilen erscheinen.
- Prüfen, dass jede 0x39-Meldung im Live System Log genau einen Eintrag belegt und keine echten Zeilenumbrüche in den Logtext einbettet.
- Bei Gimbal-Logframes prüfen, dass nur len=... und eine begrenzte Hex-Vorschau erscheinen; keine Unicode-Ersatzzeichenketten.
- Save Log auslösen und kontrollieren, dass die dekodierten Linux-Zeilen sowie kompakten Gimbal-Zeilen konsistent exportiert werden.
- Falls zukünftig [CAMERA/LITEOS] erscheint, zugehöriges USB-Log sichern; erst danach die Source-1-Kodierung festlegen.

10. Änderungshistorie

Version	Datum	Bearbeiter	Status	Änderung
v1.4	2026-10-02	OpenAI / ChatGPT	Arbeitsstand	0x39 korrekt als asynchronen Datensatz geroutet; generische Textdekodierung noch unvollständig.
v1.5	2026-10-02	OpenAI / ChatGPT	Arbeitsstand	CameraLogData-Header, Linux XOR 0x55, binäre Gimbal-Darstellung, Längvalidierung und Dokumentation ergänzt.